

Package ‘spscsf’

September 29, 2026

Title Semiparametric Smooth-Coefficient Stochastic Frontier Analysis

Version 0.1.0

Maintainer Kai Sun <ksun1@shu.edu.cn>

Description Provides semiparametric smooth-coefficient stochastic frontier analysis following Sun and Kumbhakar (2013) [<doi:10.1016/j.econlet.2013.05.001>](https://doi.org/10.1016/j.econlet.2013.05.001) where the coefficients of the parametric part vary smoothly with a set of nonparametric variables. Inefficiency term is allowed to depend on a set of determinants through heteroskedasticity. Smooth coefficients are estimated using nonparametric regression and the remaining frontier parameters are estimated by maximum likelihood. Technical efficiency and inefficiency are computed using the Battese and Coelli (1988) [<doi:10.1016/0304-4076\(88\)90053-X>](https://doi.org/10.1016/0304-4076(88)90053-X) and Jondrow et al. (1982) [<doi:10.1016/0304-4076\(82\)90004-5>](https://doi.org/10.1016/0304-4076(82)90004-5) methods, respectively. Confidence intervals for technical efficiency are computed using the approach of Horrace and Schmidt (1996) [<doi:10.1007/BF00157044>](https://doi.org/10.1007/BF00157044).

License AGPL (>= 3)

Encoding UTF-8

Depends R (>= 3.5)

Imports Formula, np, stats

Config/roxygen2/version 8.1.0

Suggests testthat (>= 3.0.0)

Config/testthat/edition 3

LazyData true

NeedsCompilation no

Author Kai Sun [aut, cre]

Repository CRAN

Date/Publication 2026-09-29 14:20:02 UTC

Contents

inefficiency	2
print.spsc_sfa	2
ricephil	3
smooth_coef	3
spsc_sfa	4
summary.spsc_sfa	8
te	9
Index	10

inefficiency	<i>Extract JLMS inefficiency estimates</i>
--------------	--

Description

Extract JLMS inefficiency estimates

Usage

inefficiency(x)

Arguments

x An object of class 'spsc_sfa'.

Value

A numeric vector of JLMS conditional inefficiency estimates.

print.spsc_sfa	<i>Print SPSC-SFA results</i>
----------------	-------------------------------

Description

Print SPSC-SFA results

Usage

```
## S3 method for class 'spsc_sfa'  
print(x, ...)
```

Arguments

x An object of class 'spsc_sfa'.
... Additional arguments (currently ignored).

Value

The original object of class "spsc_sfa", returned invisibly to avoid duplicate printing after printing its summary. The returned object is a list containing the components `smooth_coefficients`, `mle_results`, `technical_efficiency`, `inefficiency`, `model_summary`, `formula_parsed`, and `data_info`. These components contain the estimated smooth coefficients and recovered intercept, maximum-likelihood results, technical-efficiency estimates and confidence intervals, JLMS inefficiency estimates, model-specification information, parsed formula components, and data-cleaning information.

ricephil

Philippine Rice Farm Data

Description

Data used to illustrate the application of the semiparametric smooth-coefficient stochastic frontier model.

Usage

```
data(ricephil)
```

Format

A data frame with 344 observations and 17 variables.

References

Timothy J. Coelli, D.S. Prasada Rao, Christopher J. O'Donnell, and George E. Battese (2005). *An Introduction to Efficiency and Productivity Analysis*. Springer, New York.

S. Pandey, P. Masciat, L. Velasco, and R. Villano (1999). Risk analysis of a rainfed rice production system in Tarlac, Central Luzon, Philippines. *Experimental Agriculture*, 35(2), 225–237.

smooth_coef

Extract smooth coefficients

Description

Extract smooth coefficients

Usage

```
smooth_coef(x)
```

Arguments

`x` An object of class 'spsc_sfa'.

Value

The estimated smooth-coefficient matrix.

spsc_sfa	<i>Semiparametric Smooth-Coefficient Stochastic Frontier Analysis</i>
----------	---

Description

Estimates a semiparametric smooth-coefficient stochastic frontier model $y_i = \alpha(Z_i) + X_i'\beta(Z_i) + v_i - u(Z_i)$ following Sun and Kumbhakar (2013). The model allows the coefficients of the parametric part to vary smoothly with a set of nonparametric variables. Technical efficiency and inefficiency are computed using the Battese and Coelli (1988) and Jondrow et al. (1982) methods, respectively. Confidence intervals for technical efficiency are computed using the approach of Horrace and Schmidt (1996).

Usage

```
spsc_sfa(
  formula,
  data = NULL,
  ezdat = NULL,
  sig_level = 0.05,
  optim_method = "optim_BFGS",
  optim_control = list(maxit = 5000),
  stval = "automatic",
  bandwidth = NULL,
  penalty = 1e+08,
  verbose = TRUE
)
```

Arguments

formula	A formula of the form $y \sim x \mid z$ or $y \sim x \mid z \mid q$, where y is the dependent variable, x are parametric inputs, z are non-parametric variables determining smooth coefficients, and q (optional) are inefficiency determinants. When the third formula part q is omitted, the z variables also serve as inefficiency determinants.
data	Optional data frame containing all variables used in formula. If supplied, variables are extracted from this data frame. If not supplied, variables are taken from the environment of formula.
ezdat	Optional data frame containing evaluation points for the non-parametric variables z . If provided, smooth coefficients are estimated at these points; otherwise, the training data points (from z) are used.
sig_level	Significance level for confidence intervals of technical efficiency (default 0.05).
optim_method	Character string specifying the optimizer: "optim" (uses Nelder-Mead), "optim_BFGS" (uses BFGS, the default), or "nlm".

optim_control	List of control parameters passed to the optimizer.
stval	Starting values for the MLE. Use "automatic" (default) for internally computed starting values, or supply a numeric vector in which the order of its elements is δ_0 , δ_1 (if any), and σ_v^2 . See Details.
bandwidth	Optional bandwidth object (either from npscoefbw or a numeric vector) for the smooth-coefficient estimation. If NULL (default), bandwidth is selected by least-squares cross-validation (LSCV).
penalty	To ensure positivity of estimated σ_v^2 , penalty applied when σ_v^2 becomes negative during a numerical search (default 1e8).
verbose	Logical; if TRUE, progress and intermediate results are printed.

Details

The stochastic production frontier model is specified as

$$y_i = \alpha(Z_i) + X_i' \beta(Z_i) + v_i - u(Z_i),$$

where y_i can be viewed as the log of output, X_i can be viewed as a vector of the log of inputs, and Z_i is a vector of covariates. $\alpha(\cdot)$ is the intercept, and $\beta(\cdot)$ is the slope vector. Both $\alpha(\cdot)$ and $\beta(\cdot)$ are unknown functions of Z_i . The noise term $v_i \sim iidN(0, \sigma_v^2)$. The inefficiency term is $u(Z_i) = \sigma_u(Z_i)\eta_i$, where $\eta_i \sim iidN^+(0, 1)$. The random variables η_i and v_i are assumed to be independent of each other and independent of X and Z .

To guarantee positivity of $\sigma_u(Z_i)$,

$$\sigma_u(Z_i) = \exp(\delta_0 + \delta_1' Z_i).$$

These imply that

$$E(u_i | Z_i) = \sigma_u(Z_i)E(\eta_i | Z_i) = \sqrt{2/\pi} \sigma_u(Z_i) = \sqrt{2/\pi} \exp(\delta_0 + \delta_1' Z_i)$$

$$\text{and } \sigma_u^2(Z_i) = \exp[2(\delta_0 + \delta_1' Z_i)].$$

The estimation involves two steps. In the first step, estimate $\theta(Z_i)$ (Intercept (theta)), $\beta(Z_i)$, and ε_i from

$$y_i = \theta(Z_i) + X_i' \beta(Z_i) + \varepsilon_i,$$

where $\theta(Z_i) = \alpha(Z_i) - E(u_i | Z_i)$ and $\varepsilon_i = v_i - [u(Z_i) - E(u_i | Z_i)] = E(u_i | Z_i) + v_i - u(Z_i)$.

Then in the second step, use the estimated ε_i to estimate δ_0 , δ_1 and σ_v^2 from

$$\varepsilon_i = \sqrt{2/\pi} \exp(\delta_0 + \delta_1' Z_i) + v_i - \exp(\delta_0 + \delta_1' Z_i)\eta_i.$$

Finally, compute $E(u_i | Z_i) = \sqrt{2/\pi} \exp(\delta_0 + \delta_1' Z_i)$ from estimated δ_0 and δ_1 . The intercept in the original stochastic production frontier $\alpha(Z_i)$ (Intercept (alpha)) is recovered as

$$\alpha(Z_i) = \theta(Z_i) + E(u_i | Z_i).$$

See Sun and Kumbhakar (2013, page 306) for more details.

Value

An object of class "spsc_sfa" containing smooth coefficients, MLE results, technical efficiency, JLMS inefficiency, model summary, data information, and the parsed formula.

A list of class "spsc_sfa" containing:

smooth_coefficients	Smooth coefficient estimates, residuals, bandwidth, and metadata.
mle_results	MLE parameter estimates, standard errors, and log-likelihood.
technical_efficiency	Technical efficiency scores (BC method) and confidence intervals.
inefficiency	Inefficiency estimates (JLMS method).
model_summary	Summary information about the model specification.
formula_parsed	Parsed formula components.
data_info	Information about data cleaning and observations used.

Note

Results can be summarized using [summary.spsc_sfa](#). Extraction functions for smooth coefficients, technical efficiency and inefficiency are available through [smooth_coef](#), [te](#), and [inefficiency](#).

Author(s)

Kai Sun <ksun1@shu.edu.cn>

References

- George E. Battese and Tim J. Coelli (1988). Prediction of firm-level technical efficiencies with a generalized frontier production function and panel data. *Journal of Econometrics*, 38(3), 387–399.
- William C. Horrace and Peter Schmidt (1996). Confidence statements for efficiency estimates from stochastic frontier models. *Journal of Productivity Analysis*, 7(2), 257–282.
- James Jondrow, C.A. Knox Lovell, Ivan S. Materov, and Peter Schmidt (1982). On the estimation of technical inefficiency in the stochastic frontier production function model. *Journal of Econometrics*, 19(2–3), 233–238.
- Kai Sun and Subal C. Kumbhakar (2013). Semiparametric smooth-coefficient stochastic frontier model. *Economics Letters*, 120(2), 305–309.

Examples

```
set.seed(1)
n <- 100
x <- rnorm(n)
z <- runif(n)
ez <- seq(0, 1, length = 1000)
q <- rnorm(n)
y <- cos(z) + sin(z) * x + rnorm(n, 0, 0.1) -
  exp(0.1 + 0.1 * z) * abs(rnorm(n, 0, 1))
dat <- data.frame(output = y, input = x, covariate = z, ineffvar = q)
```

```

# covariate enters the nonparametric part and is also treated as inefficiency determinant
result <- spsc_sfa(output ~ input | covariate, data = dat,
                  optim_method = "optim_BFGS", verbose = TRUE)
result$smooth_coefficients
result$mle_results
result$technical_efficiency
result$inefficiency

# This gives the same results: the inefficiency variables default to the nonparametric part
result <- spsc_sfa(output ~ input | covariate | NULL, data = dat,
                  optim_method = "optim_BFGS", verbose = TRUE)

# Use evaluation data (ezdat) for estimating the smooth coefficients
result <- spsc_sfa(output ~ input | covariate, data = dat,
                  ezdat = data.frame(covariate = ez),
                  optim_method = "optim_BFGS", verbose = TRUE)
nrow(result$smooth_coefficients$theta_beta)

# Three-part formula: nonparametric part differs from inefficiency determinants
result <- spsc_sfa(output ~ input | covariate | ineffvar, data = dat,
                  optim_method = "optim_BFGS", verbose = TRUE)

# Customized bandwidth for the nonparametric part
result <- spsc_sfa(output ~ input | covariate | ineffvar, data = dat,
                  optim_method = "optim_BFGS", bandwidth = c(0.1), verbose = TRUE)

# Restrict no inefficiency determinants (intercept only)
result <- spsc_sfa(output ~ input | covariate | 1, data = dat,
                  optim_method = "optim_BFGS", verbose = TRUE)

# More variables
set.seed(1)
n <- 100
x1 <- rnorm(n)
x2 <- rnorm(n)
z1 <- runif(n)
z2 <- runif(n)
y <- cos(z1 + z2) + sin(z1 - z2) * x1 + sin(z1 + z2) * x2 +
    rnorm(n, 0, 0.1) - exp(0.1 + 0.1 * z1 + 0.1 * z2) * abs(rnorm(n, 0, 1))
dat <- data.frame(y, x1, x2, z1, z2)

result <- spsc_sfa(y ~ x1 + x2 | z1 + z2, data = dat)

# z3 is an unordered categorical variable
# spsc_sfa() detects if z3 is a factor
dat$z3 <- factor(rep(sample(c("A", "B", "C", "D", "E")), 20))

# z1, z2, z3 enter the nonparametric part and are also inefficiency determinants
result <- spsc_sfa(y ~ x1 + x2 | z1 + z2 + z3, data = dat)

# z3 is preserved as an unordered categorical variable in the nonparametric part
result$smooth_coefficients$bandwidth

```

```

# z3 is converted to dummy variables as inefficiency determinants
result$mle_results$delta_table

# Same as above (inefficiency defaults to nonparametric part)
result <- spsc_sfa(y ~ x1 + x2 | z1 + z2 + z3 | NULL, data = dat)

# Three-part formula: use z3 only as the inefficiency determinant
result <- spsc_sfa(y ~ x1 + x2 | z1 + z2 + z3 | z3, data = dat)

# Use external variables as inefficiency determinants
dat$q1 <- rnorm(n)
dat$q2 <- rnorm(n)
result <- spsc_sfa(y ~ x1 + x2 | z1 + z2 + z3 | q1 + q2, data = dat)

# No inefficiency determinants (intercept only)
result <- spsc_sfa(y ~ x1 + x2 | z1 + z2 + z3 | 1, data = dat, verbose = TRUE)

# Application with real data (ricephil)
## Not run:

data(ricephil)

ricephil$FARMERCODE <- as.factor(ricephil$FARMERCODE)

# this example shows categorical variable handling
result <- spsc_sfa(PROD ~ AREA + LABOR + NPK | EDYRS + FARMERCODE | FARMERCODE,
                  data = ricephil, optim_method = "optim_BFGS", verbose = TRUE)

# FARMERCODE is preserved as an unordered categorical variable in the nonparametric part
result$smooth_coefficients$bandwidth

# FARMERCODE is converted to dummy variables as inefficiency determinants
result$mle_results$delta_table

## End(Not run)

```

summary.spsc_sfa

Summary method for SPSC-SFA

Description

Summary method for SPSC-SFA

Usage

```

## S3 method for class 'spsc_sfa'
summary(object, ...)

```

Arguments

object An object of class 'spsc_sfa'.
... Additional arguments (currently ignored).

Value

See [print.spsc_sfa](#).

te	<i>Extract technical-efficiency scores</i>
----	--

Description

Extract technical-efficiency scores

Usage

`te(x)`

Arguments

x An object of class 'spsc_sfa'.

Value

A numeric vector of technical-efficiency estimates.

Index

* datasets

ricephil, [3](#)

inefficiency, [2](#), [6](#)

print.spsc_sfa, [2](#), [9](#)

ricephil, [3](#)

smooth_coef, [3](#), [6](#)

spsc_sfa, [4](#)

summary.spsc_sfa, [6](#), [8](#)

te, [6](#), [9](#)